

BLOCO 2

O Pipeline de Dados e Busca

Da bagunça documental ao contexto semântico
preciso



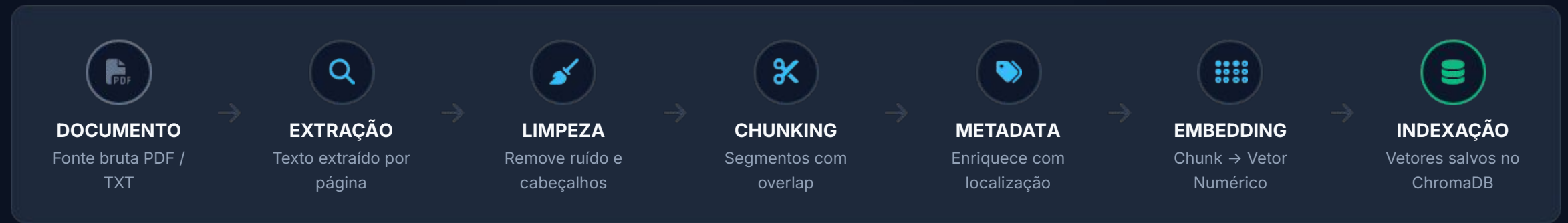
Cobre: Ingestão & Indexação + Recuperação Semântica

Unidades 3 e 4 · 60 min · RAG Foundations Course ·
ETIPI

600 × 400

Fase Offline: o ETL dos seus documentos

💬 Você já fez ETL. Extração → Transformação → Carga. Isso é exatamente o que o pipeline de ingestão faz — só que o destino é um banco vetorial, não um DW.



← Nunca misturar com a fase online Roda offline, uma vez (ou por atualização) →

Referências: `src/1_ingestion.py` · `src/2_indexing.py`

PDFs institucionais: o inimigo dos embeddings

O que parece no PDF (Visual):

Universidade Federal — Pró-Reitoria de Graduação
Resolução nº 023/2025 — Página 14 de 47

Art. 14. O prazo máximo para solicitação de revisão de nota é de 10 (dez) dias úteis após a divulgação dos resultados...

Universidade Federal — Pró-Reitoria de Graduação
Resolução nº 023/2025 — Página 14 de 47

O que o extrator vê (Texto Bruto):

Texto bruto extraído sem limpeza prévia:

"Universidade Federal – Pró-Reitoria de Graduação
Resolução nº 023/2025 – Página 14 de 47"

Art. 14. O prazo máximo para solicitação de revisão de nota é de 10 (dez) dias úteis após a divulgação dos resultados...

Universidade Federal – Pró-Reitoria de Graduação
Resolução nº 023/2025 – Página 14 de 47"

⚠ **Cabeçalhos e rodapés repetidos envenenam o embedding.** O chunk fica com 40% de ruído institucional e 60% de conteúdo real. O embedding captura ambos, e a busca semântica falha.

Extração: texto por página, não o documento inteiro

⚠ Nunca extraia o PDF como uma string única gigante. Extrair página por página preserva a localização — e **localização é metadado crítico**.

src/1_ingestion.py

```
import pypdf

def extract_pages(pdf_path: str) -> list[dict]:
    pages = []
    with open(pdf_path, "rb") as f:
        reader = pypdf.PdfReader(f)

        for i, page in enumerate(reader.pages, start=1):
            raw_text = page.extract_text() or ""

            pages.append({
                "page": i,          # ← número metadado
                "raw_text": raw_text # ← texto bruto
            })

    return pages
```

- ↳ `enumerate(..., start=1)`:
A página começa em 1, não em 0 — correspondendo exatamente ao que o usuário final vê no leitor de PDF.
- ↳ `page.extract_text()` or `""`:
Páginas vazias (ex: capas imagem) retornam `None`. O fallback vazio evita crashes no pipeline.
- ↳ `"page": i`:
Este número acompanha o chunk até o fim e aparecerá na citação gerada pelo LLM.
- ↳ `"raw_text"`:
Guardamos o texto bruto. A limpeza rigorosa acontecerá no próximo passo do pipeline.

Limpeza: texto sujo gera embedding sujo

Antes da limpeza

"Universidade Federal – Pró-Reitoria\nde Graduação\n\n\n Art. 14. O prazo máxi-\nmo para solici-\ntação de revisão\nd00adta é de 10 dias"

Função de Normalização

```
import re

def normalize_text(text: str) -> str:
    # 1. Remove soft-hyphen invisível (\u00ad) – PDFs adoram inserir isso
    text = text.replace("\u00ad", "")
    # 2. Junta palavras quebradas por hífen no fim de linha (ex: solici-\ntação)
    text = re.sub(r"(\w)-\s+(\w)", r"\1\2", text)
    # 3. Colapsa qualquer sequência de espaços/tabs/newlines em espaço único
    return re.sub(r"\s+", " ", text).strip()
```

Depois da limpeza

"Art. 14. O prazo máximo para solicitação de revisão de nota é de 10 dias úteis após a divulgação..."

O \u00ad (soft-hyphen) não aparece na tela do usuário, mas aparece no embedding. É invisível para humanos e tóxico para a IA.

Chunking: sliding window sobre o texto

É literalmente um algoritmo de *sliding window* sobre uma string grande. Você provavelmente já implementou algo semelhante para processar arrays ou streams de dados de forma iterativa.

src/1_ingestion.py

```
def chunk_with_overlap(text: str, size=1200, overlap=200):
    chunks = []
    start = 0
    while start < len(text):
        # Corta o bloco do tamanho estipulado
        chunks.append(text[start : start + size])

        # Avança a janela (tamanho - overlap)
        start += max(1, size - overlap)

    return chunks
```

Texto completo da página normalizada:

Art. 13... Art. 14... Art. 15...

↓ chunk_with_overlap()

Art. 13... Art. 14... 0 → 1200

Art. 14... 0 prazo... 1000 → 2200

Art. 15... 2000 → 3200

↑ O **overlap** (200c) garante que o contexto não sofra cortes secos

Tamanho do chunk: a decisão de maior impacto

Não existe tamanho universalmente correto. Existe o tamanho certo para o seu tipo de documento e caso de uso específico.

Tamanho	Exemplo de conteúdo	Embedding resultante	Impacto no retrieval	Rec?
~200 chars	"O prazo é de 10 dias."	Muito específico, perde contexto amplo	Recupera frases soltas, resposta fica incompleta	✗
	Parágrafo completo	Semântica clara e coesa do assunto	Recupera contexto suficiente para o LLM raciocinar	✓
~5000 chars	Vários artigos de temas diferentes agrupados	Embedding "diluído" por múltiplos temas conflitantes	Busca semântica retorna o chunk errado frequentemente	✗

☰ **Heurísticas de partida:**

- > **Textos corridos:** 1000–2000 chars (overlap 10-20%)
- > **Docs estruturados:** Chunking semântico por seção/artigo
- > **Em produção:** Meça em tokens, não chars (1200c ≈ 300t)

⚠ **Cuidado:** Um chunk de 5000 chars não é "mais rico". É um vetor que mistura assuntos. O LLM precisa de precisão, não volume. Prefira chunks focados e um Top-K maior.

Metadados: governança embutida em cada chunk

💬 Pense como um *covering index* relacional. O embedding é a chave de busca. A metadata é o payload retornado junto — sem custo extra de JOIN, pois já reside no Vector DB.

Objeto Chunk Estruturado

```
chunk_obj = {  
  "chunk_id": "reg2025:p14:03", # PK única  
  "text": "Art. 14. O prazo...",  
  
  "metadata": {  
    # 📌 Citação (Para a resposta)  
    "source": "Regimento 2025",  
    "page": 14,  
    "section": "Avaliação",  
  
    # 🔒 Governança (Auditoria)  
    "doc_hash": "sha256:a3...",  
    "version": "v1.2",  
  
    # 🔍 Filtros (WHERE clause)  
    "doc_type": "regimento",  
    "year": 2025  
  }  
}
```

- ↳ chunk_id no topo:
É a chave primária de junção no índice vetorial.
- ↳ Grupo source / page:
Esses campos acompanharão o texto e comporão a citação clicável na resposta final do LLM.
- ↳ Grupo hash / version:
Permitem detectar versões inconsistentes no banco. Quando um PDF é atualizado, usamos o hash para invalidar chunks velhos.
- ↳ Grupo type / year:
São as chaves vitais utilizadas no WHERE (filtro pré-busca) durante a execução da query.

Sem metadata, o RAG é uma caixa preta inauditaável

Metadata não entra no vetor numérico — mas é o que torna o RAG institucional viável em produção.



Busca Cirúrgica (Filtrar)

Buscar apenas em documentos vigentes de 2025, do tipo "resolução".

`where={"year": 2025, "type": "resolucao"}`



Rastreabilidade (Citar)

A resposta gerada cita sua origem: "*Fonte: Regimento 2025 | pág. 14*". O usuário pode clicar e validar contra a fonte oficial original.



Trilha de Acesso (Auditar)

Os logs do sistema registram qual chunk foi usado para embasar cada resposta enviada. Exigência fundamental de Compliance e LGPD.



Reindexação Segura (Versionar)

`pipeline_version` + `doc_hash` garantem a identificação exata de qual documento gerou aquele vetor, evitando bancos de dados "sujos".

⚠ "Não preciso de metadata no protótipo" → O protótipo vai para produção. Custo zero para adicionar agora. Custo altíssimo para adicionar depois (reindexação total).

Indexação: persistência atômica no ChromaDB

Com os chunks segmentados e enriquecidos com metadados, o passo final da Fase Offline é gravá-los de forma persistente no banco de dados vetorial.

src/2_indexing.py

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(
    name="institutional_docs",
    metadata={"hnsw:space": "cosine"} # ← métrica vetorial
)

def index_chunks(chunks: list[dict]) -> None:
    collection.add(
        ids = [c["chunk_id"] for c in chunks], # PK
        documents = [c["text"] for c in chunks], # Texto
        embeddings= [c["embedding"] for c in chunks], # Vetor
        metadatas = [c["metadata"] for c in chunks] # JSON
    )
```

- ↳ PersistentClient:
Dados são salvos fisicamente em disco, sobrevivendo a reinicializações do servidor.
- ↳ hnsw:space: cosine:
A métrica geométrica que o banco usará. Definida na criação, não pode ser alterada.
- ↳ Arrays Paralelos em .add():
O banco vetorial não usa tabelas relacionais. O vínculo ocorre porque o índice [i] da lista de IDs pertence ao índice [i] da lista de Textos, Vetores e Metadados. Tudo armazenado e recuperado junto.

Na prática: o pipeline offline rodando

Terminal - bash

```
$ python src/1_ingestion.py data/regimento.pdf
```

RAG Foundations – Ingestão de Documentos

📄 Documento: regimento.pdf (47 páginas)

Extração ██████████ 47/47 ✓

Chunking ██████████ 203 chunks ✓

Cabeçalhos removidos: 94 ocorrências

```
$ python src/2_indexing.py
```

Gerando embeddings ██████████ 172/203

ETA: ~28s | nomic-embed-text @ localhost:11434

Indexando no ChromaDB... ✓

✅ **203 chunks indexados com sucesso**

DB: ./chroma_db | Dim: 768 | Duração: 43.2s

47 págs → 203 chunks

Com janela de 1200 chars e overlap de 200, geramos uma média de ~4,3 chunks matemáticos por página original.

localhost:11434

Utilizando o Ollama rodando localmente. Zero tráfego de rede externo ou chamadas de API cloud pagas durante toda a ingestão institucional.

FASE ONLINE

Recuperação

Do banco vetorial ao contexto entregue ao LLM

Doc → Extração → Chunk → Index

[Online: → Agora]

Pergunta → Busca → Contexto → LLM

↑ Estamos aqui

minístico que o modelo.
ieval primeiro."

Por que retrieval importa mais do que o modelo

O LLM é estocástico. O retrieval é determinístico — dada a mesma pergunta e índice, **sempre retorna os mesmos chunks**. Erros de retrieval são rastreáveis e resolvíveis.

Erro de Retrieval

Chunk *errado* recuperado → LLM responde com base em evidência errada.

- > **Causa:** Chunking ruim, embedding fraco, filtros ausentes.
- > **Diagnóstico:** Inspeção os chunks do top-K diretamente (ignore a IA).
- > **Correção:** Ajuste o pipeline offline → reindexe.

Erro de LLM / Prompt

Chunk *correto* recuperado → LLM ignora a evidência e alucina.

- > **Causa:** Prompt fraco sem instrução de grounding rígida.
- > **Diagnóstico:** Compare a resposta com o texto exato do chunk.
- > **Correção:** Ajuste o prompt na requisição online da API.

Dúvida de onde está o erro? Inspeção o top-K retornado primeiro. Sempre.

Top-K: a query do banco vetorial

```
</> SELECT * FROM chunks ORDER BY cosine_similarity(chunk, query) DESC LIMIT k
```

Pipeline de Busca

1. Pergunta do usuário

↓ `embed_text(query)`

↓ `collection.query(`

`n_results=TOP_K`

`)`

↓ Chunks ordenados (Distância)

★ QUERY: "prazo para pedir revisão"

K = 1 "Art. 14 — revisão de nota"

K = 2 "Art. 15 — prazo recurso"

K = 3 "Art. 16 — aprovação"

K = 4 "resolução 001 capa" ⚠ Corte

K = 5 "ata reunião geral" ❌ Corte

No código: como a busca semântica é executada

src/3_retrieval.py

```
def retrieve(query: str, top_k: int = 5, filt_src=None) -> list:
    # 1. Embedding da pergunta (CRÍTICO: Mesmo modelo)
    query_emb = embed_text(query)

    # 2. Executa a busca vetorial (ANN)
    res = collection.query(
        query_embeddings = [query_emb],
        n_results = top_k,
        where = {"source": filt_src} if filt_src else None,
        include = ["documents", "metadatas", "distances"]
    )

    # 3. Monta lista estruturada
    hits = []
    for doc, meta, dist in zip(res["documents"][0], ...):
        hits.append({
            "text": doc, "metadata": meta,
            "similarity": 1 - dist # Inverte distância
        })
    return hits
```

- ↳ `embed_text(query)`:
Obrigatório usar o exato mesmo modelo de IA da ingestão.
- ↳ `where={...}`:
Executa filtros exatos em metadados antes de ordenar geometricamente.
- ↳ `include=[...]`:
ChromaDB retorna Texto, Metadados e Distância no mesmo DTO.
- ↳ `1 - dist`:
Cosseno no Chroma retorna *Distância* (0=idêntico).
Invertemos para exibir *Similaridade* em % para o usuário.

Filtros por metadata: o WHERE clause do RAG

❌ Sem filtro (Busca livre / Risco de alucinação)

```
# Busca em TODO o índice - incluindo
# normas revogadas, versões antigas.

res = collection.query(
    query_embeddings=[emb],
    n_results=5
    # Risco: Pode retornar a Resolução 2020
    # que foi revogada pela de 2025!
)
```

✅ Com filtro (Busca cirúrgica contextualizada)

```
# Busca apenas em documentos vigentes
# de 2025, especificamente do tipo "resolução".

res = collection.query(
    query_embeddings=[emb],
    n_results=5,
    where={"year": 2025, "type": "resolucao"}
    # Garante evidências confiáveis para a IA
)
```

Campo filtrado	Caso de uso institucional RAG
year	Evitar o uso de normas e manuais desatualizados no raciocínio.
doc_type	Separar documentos canônicos (regras) de não-canônicos (atas).
status: "vigente"	Ação Crítica: Excluir mecanicamente documentos revogados do contexto.

Top-K: como escolher sem chutar (Trade-offs)

K	Cobertura (Recall)	Ruído (Precision)	Custo de tokens	Uso recomendado
k = 3	▲ Baixa	✓ Baixo	✓ Econômico	Chunks grandes, perguntas literais e pontuais
✓	✓ Boa	✓ Controlado	✓ Equilibrado	Padrão ótimo inicial (Usado neste curso)
k = 8	✓ Alta	▲ Cresce	▲ Médio	Perguntas amplas exigindo síntese de várias fontes
k = 30+	✓ Máxima	✗ Muito Alto	✗ Muito Caro	Apenas e estritamente como input para Re-ranking

A Regra de Ouro da Arquitetura de Busca (Avançado):

Recuperar muito → Re-ranquear → Enviar pouco e bom.
Exemplo: Busca Vetorial k=30 → Cross-Encoder Re-ranker → k=5 enviados ao Prompt.

Re-ranking: quando "parecido" não é suficiente

💬 Padrão idêntico ao *candidate generation + ranking* de sistemas como Netflix. Um passo leve (vetores) gera a shortlist. Um passo pesado (cross-encoder) reordena ela perfeitamente.

PASSO 1: Vetorial (ANN)

Top 30 Candidatos (Rápido)
[Chunk A: 0.89]
[Chunk B: 0.87] ← *ordem imprecisa*
[Chunk C: 0.85]



PASSO 2: Cross-Encoder

Re-avalia (Pergunta + Chunk) juntos
Chunk C → 0.97 ↑ 1°
Chunk A → 0.91 ↓

Métrica	Busca Vetorial (Bi-Encoder)
Compara	Query e doc calculados separados
Velocidade	⚡ Muito Rápido (5-20ms)
Uso típico	Gerar a shortlist (Alto Recall)
Métrica	Re-ranker (Cross-Encoder)
Compara	Query e doc calculados juntos (deep attention)
Velocidade	🐢 Mais Lento (50-200ms)
Uso típico	Ordenar a shortlist (Alta Precision)

Três abordagens de re-ranking

Abordagem	Qualidade	Custo / Tempo
Cross-Encoder Rede neural dedicada para score	✓ Máxima	⚠ Médio
LLM Re-ranking Prompt pede para o LLM ordenar	✓ Boa	✗ Muito Alto
Rank Fusion (RRF) Combinação Vetorial + BM25	✓ Boa	✓ Baixo

Snippet: Cross-Encoder

```
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("ms-marco-MiniLM-L-6-v2")

def rerank(query: str, candidates: list, top_n=5):
    # Monta pares (pergunta, documento)
    pairs = [(query, c["text"]) for c in candidates]

    # O modelo prevê score semântico relacional
    scores = reranker.predict(pairs)

    # Ordena pelo novo score semântico
    ranked = sorted(zip(candidates, scores),
                    key=lambda x: x[1], reverse=True)

    return [chunk for chunk, _ in ranked[:top_n]]
```

O que você agora sabe (Checklist do Bloco)

🔹 Ingestão (Offline)

- ✔ Extrair PDF por página preserva a localização.
- ✔ Texto sujo destrói o vetor. Normalizar é regra.
- ✔ Chunking com tamanho errado destrói o recall.
- ✔ Metadata não entra no vetor, mas torna o RAG auditável e apto para produção.

🔍 Recuperação (Online)

- ✔ Top-K é determinístico. Inspecione a query.
- ✔ Filtro de metadata atua como o WHERE clause.
- ✔ Top-K alto → mais Recall.
Re-ranking → mais Precision.
- ✔ A similaridade do cosseno compara ângulos semânticos.



Próximo passo — Bloco 3: Integração e Geração O contexto foi recuperado com precisão. Agora: como transformar esses chunks em uma resposta confiável — o prompt RAG, janela de contexto, integração de API e avaliação prática do sistema.